The Walking Dead-Ends: Avoiding Local Optima in TD3-based Bipedal Learning

Joshua Bourton. CIS Username: xtzv61

Abstract

This paper presents a Bipedal Walker solution based on the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm for continuous control in the rldurham Walker environment [1]. The final agent achieves quick, stable convergence in easy mode, gaining consistent and regular episodic rewards above 220 with very little variance in approximately 200 training episodes, and demonstrates rapid learning in hardcore mode with a learning schedule applied, exceeding episodic rewards of 210 within 600 episodes.

1 Methodology

To solve the rldurham bipedal walker challenge, I updated provided SAC coursework template to use the recommended TD3 approach [1], which addresses overestimation bias in actor-critic methods by combining clipped double Q-learning, delayed policy updates and target policy smoothing. CleanRL's reference implementation was used as a conceptual baseline upon which I built my own variant and incorporated the training and stabilisation modifications described in this paper [2]. Although the core implementation is generally consistent with standard TD3, my solution does include multiple additional mechanisms, including n-step return estimation, action smoothing and curriculum-based episode length scaling, to improve training stability in the Walker environment.

1.1 Constructing a baseline in hardcore=False

Using CleanRL's TD3 implementation as the conceptual reference, I first constructed a computationally inexpensive baseline for quick, multi-seed experimentation, with the objective of establishing a stable configuration in `hardcore=False` mode prior to progressing into `hardcore=True`. Layernorm to stablise gradients in early training as per [3] was included in this baseline. The primary hyperparameters are shown in Table 1. To ensure comparability, I determined the success of each significant experimental change using using three random seeds, ran each ablation for 1000 episodes and assessed learning behaviour through the shapes of the episodic reward statistics (r_{mean}, r_{std} and r_{sum}).

Table 1: Initial TD3 baseline configuration used for preliminary experiments.

Batch	γ	Actor LR	Critic LR	Delay	σ	Warmup	n-step	α_{smooth}
128	0.99	3×10^{-4}	3×10^{-4}	2	0.1	0	n/a	n/a

Under this baseline configuration, the agent converged to an episodic reward of ~ 170 between episodes 300 and 400. However, performance remained unstable after convergence, with episodic reward fluctuating substantially and occasionally dropping to values nearby ~ -100. This indicated a viable locomotion policy but revealed the need for hyperparameter tuning and additional upgrades.

Therefore I conducted a series of experiments to improve convergence speed and training stability. To start, increasing batch size from 128 to 256 gave slightly faster convergence. Expanding replay buffer capacity from 50,000 transitions to 1M and subsequently 3M gave sequential modest improvements in convergence speed, so 3M was kept. Training lag was reduced by replacing the episodic update scheme, whereby all gradient updates ran after each

full episode, with step-wise online updates, where a single gradient step was performed after each environment interaction. This gave the critics continuously refreshed loss landscapes at the cost of slightly higher variance due to higher correlation between successive updates. Reward scaling with a factor of 10 was also briefly evaluated but removed as it significantly prolonged convergence without improving final performance.

I examined exploration behaviour using both static and decaying noise schedules. Several exploration strategies inspired by prior continuous-control work [4] were evaluated, revealing that large initial noise values combined with slow decay schedules hinder early learning, whereas lower initial noise values (e.g. 0.15–0.25) with decay rates around 0.99 and minimum floor 0.1 produced more stable post-convergence behaviour. Exploration noise used a multiplicative episode-based decay schedule form $\sigma_t = \max(\sigma_{\min}, \sigma_0 \cdot \lambda^t)$, where σ_t is the noise magnitude applied at episode t, σ_0 the initial noise level, λ the decay factor and $\sigma_{\min}$ a fixed lower bound preventing noise from decaying to zero. This allowed exploration to gradually reduce throughout training whilst maintaining a small degree of variability for continual policy refinement. However, further experiments showed a fixed static noise of 0.2 produced similar stability results, and thus this technique was discarded.

Architectural capacity was investigated by increasing actor and critic hidden layer widths from 256 to 512 units per layer, which yielded negligible improvements but increased training time. Increasing batch size for this larger architecture didn't give measurable performance gains. Then I added a warmup phase whereby transitions were collected before gradient updates commenced, with the canonical 10,000 environment steps chosen for stability [1]. Environment and reward normalisation as in [5] were introduced but brought substantial instability so were discarded. Further architectural refinements removed layer normalisation and introduced orthogonal weight initialisation with small final-layer initialisation for both actor and critic networks [6]. Critic gradient clipping was also incorporated, and several critic learning rates were evaluated, with 3×10^{-4} producing the most consistent results.

Finally, several additional modifications were evaluated but ultimately discarded after failing to improve performance relative to the evolving baseline. These included reducing training frequency, alternative policy delay values, reward clipping [7], actor gradient clipping and adjustments to action scaling parameters.

The resulting new baseline architecture achieved a maximum episodic reward of approximately ~ 229 and showed visible fast, stable forward running behaviour. Some variance remained across seeds but overall the architecture (with primary hyperparameters shown in 2 provided a reliable platform for which to take on the `hardcore=True` environment.

Table 2: Final TD3 configuration used for the submitted hardcore=False agent.

Batch	γ	Actor LR	Critic LR	Delay	σ	Warmup	n-step	α_{smooth}
256	0.99	3×10^{-4}	3×10^{-4}	3	0.2	10,000	*n/a*	*n/a*

1.2 Refining architecture on hardcore=True

Running the new baseline model on `hardcore=True` for 2000 episodes produced a rolling mean curve below 0, making clear that the architecture was insufficient to teach locomotion on the trickier terrain. To tackle this, I experimented with a collection of techniques from the literature throughout multiple 3-seed ablations, eventually achieving an agent capable of frequently completing runs with fast and stable locomotion.

My first major update was implementation of n-step return estimation [8]. Instead of bootstrapping from a singular transition, the n-step return accumulates discounted rewards over n steps before bootstrapping from the n-th successor state:

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n Q(s_{t+n}, \pi(s_{t+n})) \tag{1}$$

I used a sliding dequeue of length n for accumulating transitions, which had a tail-flush mechanism at episode boundaries so that partial n-step transitions weren't discarded. To

handle transitions shorter than n steps (i.e. near episode termination), I used a dynamic discount of $\gamma^{n_{\text{used}}}$ for each sample instead of fixed γ^n. Values of $n \in \{3, 5, 7\}$ were trialed, with $n = 3$ adopted as it produced similar early improvement but with the lowest variance at convergence.

My second upgrade, inspired by [9], was episode length scheduling for scaffolding training difficulty. Initially I used short episode caps to encourage focus on basic locomotion before experiencing hardcore's full complexity. I trialed eight different schedules with varying episode thresholds and caps, settling on the schedule in Table 3, which produced an upward trend from episode 1000, mean reward 0 to ~ 150, and max reward ~ 170.

Table 3: Best-performing Episode Length Schedule

Episode number	**Episode length cap**
0–149	300
150–399	600
400–699	1000
≥ 700	No cap

Next, I implemented action smoothing in an attempt to reduce erratic movements and thus stablise the policy. I did this with Exponential Moving Average (EMA) smoothing over consecutive actions, which I applied prior to adding exploration noise:

$$a_t = \alpha \cdot \pi(s_t) + (1 - \alpha) \cdot a_{t-1} \tag{2}$$

I tried α values in range $\in \{0.25, 0.50, 0.90, 0.93, 0.95, 0.97\}$; $\alpha = 0.93$ was adopted as it produced the best convergence curve on hardcore, achieving mean rewards exceeding 170 and max reward reaching ~ 190 by approximately episode 1600. Lower values ($\alpha = 0.25, 0.50$) smoothed excessively, distorting policy signal. $\alpha = 0.97$ was marginally worse, likely over-constraining action variation during early exploration.

I also refined the target policy smoothing noise parameters for hardcore. Pairs $(\sigma_{\text{noise}}, c) \in \{(0.10, 0.25),\ (0.15, 0.30),\ (0.18, 0.30),\ (0.20, 0.30),\ (0.24, 0.36)\}$ were tested, where c is the clip bound applied to the sampled noise $\varepsilon \sim \text{clip}(\mathcal{N}(0, \sigma_{\text{noise}}), \pm c)$ [1]. I kept the pair $(0.18, 0.30)$ as they produced the smoothest convergence curve across seeds. Additionally, I reintroduced reward clipping to [-10, 10] [7] as it helped combat extreme fall penalties.

I tried but ultimately discarded a handful of other techniques. One of these was the TD3 + Behaviour Cloning regularisation technique [10], in which actor loss is augmented with a behavioural cloning term weighted by a normalised Q-value scale. However, this resulted in slower convergence and lower mean episodic rewards, likely due to the algorithm being designed for offline agents whereas this was online without demonstration data to anchor itself and therefore biased the policy away from exploratory actions. Inspired by [11], I also trialed Prioritised Experience Replay (PER) [12], whereby I replaced uniform buffer sampling with weighted sampling proportional to each transition's temporal-difference error, and used canonical importance-sampling correction to remove resulting bias. However, it gave a downward episodic mean reward trend, possibly because importance-sampling correction interacted badly with my update regime's fast-moving targets.

I next attempted applying a CAPS smoothness penalty to actor loss per [13] by augmenting my standard policy gradient with a term that penalises the norm of the difference between actions produced by nearby states to encourage policy spatial consistency. This too stagnated, this time below zero mean reward. I trialed a third critic head, LayerNorm variants, and combinations of both, which all failed to improve over the two-critic baseline. Reducing training frequency to one gradient step every two environment steps, survival cost caps and critic Huber loss also caused performance degradation. The final configuration used for hardcore mode is summarised in Table 4.

Table 4: Final TD3 configuration used for the submitted hardcore=True agent.

Batch	γ	Actor LR	Critic LR	Delay	σ	Warmup	n-step	α_{smooth}
256	0.99	3×10^{-4}	3×10^{-4}	3	0.2	10,000	3	0.93

2 Convergence Results

The final architectures used to solve both hardcore and easy mode were similar less the architectural features introduced in section 1.2 including learning schedule, which was excluded from easy mode due the relative low difficulty of the hardcore=False environment as it hindered early learning and caused higher variance. In hardcore=False, the agent learned a viable locomotion strategy exceeding episodic reward of ∼220 after about 200 episodes, after which performance remained wholly stable over the 1000 episode run-time, less a few bad runs (Fig.1).

A broadly similar but less pronounced pattern is present in hardcore performance. Per Fig.2, the agent consistently achieves a fast and stable gait across many runs, with individual episodes frequently reaching the maximum episodic reward of ∼210 by approximately episode 600. It shows a clear three-phase learning trajectory due to the aforementioned episode length schedule, whereby during the early learning stages (episodes 0–700), episode length caps are progressively lifted and available episodic reward increases. After full uncapping, max reward plateaus as the agent settles into its sustained improvement phase, with mean episodic reward gently rising but individual episode runs fluctuating significantly between −100 and +210 due to the per-episode obstacle diversity in hardcore's terrains.

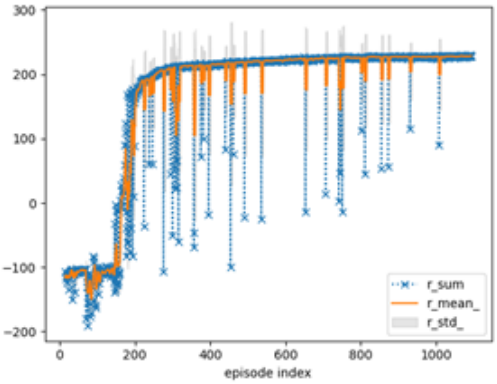


Figure 1: `hardcore=False` model.

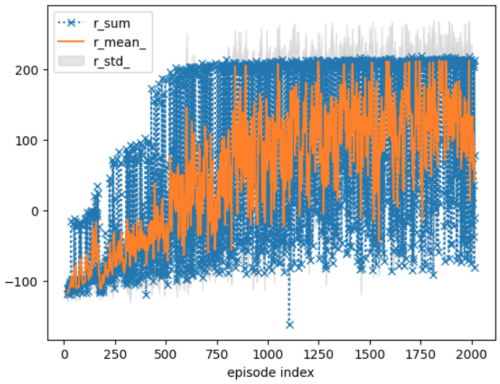


Figure 2: `hardcore=True` model.

3 Limitations and Future Work

The final agent's main limitation is its persistently high episodic variance; whilst many runs perform strongly, some end in a fall, likely because of out-of-distribution terrain configurations with tricky random obstacle placements. Thus, future work should focus on reducing variance in hardcore, potentially trying curiosity-driven exploration [14], which provides intrinsic reward proportional to the agent's prediction error on novel states to encourage exploration of unfamiliar terrain configurations. Population-based training [15], which jointly optimises a population of models by asynchronously replacing under-performing networks with the top-performing one's weights to continuously update hyperparameters, could also be trialed to identify high-performing, low-variance configurations.

Another quite major limitation was the CPU-bound training, as it constrained the total possible ablations due to 4-6hr runs. Future work should investigate GPU-accelerated training to see if equivalent convergence can be achieved in quicker wall-clock time to enable more extensive hyperparameter exploration. Similarly, hyperparameters were tuned in sequential single-variable ablations, which inherently can't detect interactions between other jointly sensitive hyperparameters. Potentially Bayesian optimisation [16] could be explored, which treats hyperparameter optimisation as a structured search problem, constructing a probabilistic model of the algorithm's performance and exploiting it to make decisions about where next to evaluate. Complemented with GPU-accelerated training, this could make an extensive hyperparameter space exploration very feasible.

Use of GenAI

The same model, ChatGPT 5.3, was used for each task mentioned in this section. GenAI was used for:

- Educating me on the layout of the provided starter template (prompt provided below).
- Suggesting valid hyperparameters, including for suggesting the aformentioned episode length schedules trialed (prompt provided below) and the noise schedules.
- Formatting this report in LaTeX, specifically the equations and tables used.

Prompt used for understanding the provided starter template: *"Examine the attatched notebook uploaded as a python file, which presents an imperfect solution for solving a modified version of the bipdal walker V3 challenge. Go for a top-down approach, starting with a high level summary, and then, for each core segment in that notebook, provide me a breakdown of what's happening in that segment, including core architectural features and major reinforcement learning concepts."*

Prompt for creating candidate episode length schedules: *"I am running a TD3 agent for 2000 episodes in hardcore mode on bipedal walker. I have decided to use episode length scheduling to try improve current performance by giving the agent a kind of stabiliser in the early stages so it can focus on learning the basics of locomotion before it tackles the whole tricky environment, and I need your help to create viable candidate episode length schedules that encompass trials that are both conservative and aggressive, alongside being somewhere inbetween. Suggest to me values of episode thresholds and caps per threshold. I believe 4 different tiers are a suitable level of complexity for warming up the agent. Present your suggestions in a table."*

Note that similar prompts were used for finding reasonable trial values for other hyperparameters, but those exact prompts are omitted.

References

[1] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International Conference on Machine Learning*, PMLR, 2018, pp. 1587–1596.

[2] S. Huang and contributors, *Td3_continuous_action_jax.py*, `https://github.com/vwxyzjn/cleanrl/blob/master/cleanrl/td3_continuous_action.py`, CleanRL GitHub repository, accessed Feb. 13, 2026, 2025.

[3] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[4] J. Hollenstein, S. Auddy, M. Saveriano, E. Renaudo, and J. Piater, "Action noise in off-policy deep reinforcement learning: Impact on exploration and performance," *arXiv preprint arXiv:2206.03787*, 2022. arXiv: `2206.03787 [cs.LG]`. [Online]. Available: `https://arxiv.org/abs/2206.03787`.

[5] L. Engstrom et al., "Implementation matters in deep policy gradients: A case study on ppo and trpo," *arXiv preprint arXiv:2005.12729*, 2020. arXiv: `2005.12729 [cs.LG]`. [Online]. Available: `https://arxiv.org/abs/2005.12729`.

[6] A. M. Saxe, J. L. McClelland, and S. Ganguli, "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks," *arXiv preprint arXiv:1312.6120*, 2013. arXiv: `1312.6120 [cs.LG]`. [Online]. Available: `https://arxiv.org/abs/1312.6120`.

[7] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. DOI: `10.1038/nature14236`. [Online]. Available: `https://doi.org/10.1038/nature14236`.

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd. Cambridge, MA: MIT Press, 2018. [Online]. Available: `https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf`.

[9] S. Bajamahal, *Learning to walk with reinforcement learning*, https://medium.com/@sunny.bajamahal/learning-to-walk-with-reinforcement-learning-191560eaa283, Medium, accessed Feb. 21, 2026, 2025.

[10] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," *arXiv preprint arXiv:2106.06860*, 2021. arXiv: 2106.06860 [cs.LG]. [Online]. Available: https://arxiv.org/pdf/2106.06860.

[11] Y. Lin, Z. Zhang, Y. Tan, H. Fu, and H. Min, "Efficient td3 based path planning of mobile robot in dynamic environments using prioritized experience replay and lstm," *Scientific Reports*, vol. 15, no. 1, May 2025. DOI: 10.1038/s41598-025-02244-z. [Online]. Available: https://doi.org/10.1038/s41598-025-02244-z.

[12] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, 2016. arXiv: 1511.05952 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1511.05952.

[13] S. Mysore, B. Mabsout, R. Mancuso, and K. Saenko, "Regularizing action policies for smooth control with reinforcement learning," *arXiv preprint arXiv:2012.06644*, 2020. arXiv: 2012.06644 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2012.06644.

[14] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," *arXiv preprint arXiv:1705.05363*, 2017. arXiv: 1705.05363 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1705.05363.

[15] M. Jaderberg et al., "Population based training of neural networks," *arXiv preprint arXiv:1711.09846*, 2017. arXiv: 1711.09846 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1711.09846.

[16] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," *arXiv preprint arXiv:1206.2944*, 2012. arXiv: 1206.2944 [stat.ML]. [Online]. Available: https://arxiv.org/abs/1206.2944.